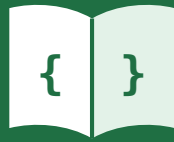

Modül 17 — GraphQL ve WebSocket

Ders Notları

Spring Boot Edu



2026-09-24

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Tek Sayfada GraphQL	2
2.2 Schema-First	3
2.3 N+1 Problemi	3
2.4 WebSocket ve STOMP	3
3. Adım Adım Örnekler	3
3.1 Şema	4
3.2 Sorgular: @QueryMapping	5
3.3 İç İç Alanlar: @SchemaMapping ve @BatchMapping	6
3.4 Mutation'lar ve Hatalar	7
3.5 Subscription'lar	8
3.6 STOMP: Canlı Fiyat Bildirimleri	8
3.7 Test	9
4. Sık Yapılan Hatalar ve En İyi Pratikler	10
5. Özet	10
6. İleri Okuma	10

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- GraphQL'in REST'ten ne zaman daha uygun olduğunu ve ne zaman olmadığını açıklamak
- Önce bir GraphQL şeması yazmak ve onu @QueryMapping, @MutationMapping ve @SchemaMapping ile gerçekleştirmek
- N+1 problemini tanımak ve @BatchMapping (bir DataLoader) ile çözmek
- Exception'ları @GraphQLExceptionHandler ile kullanışlı GraphQL hatalarına çevirmek
- İstemcilere veri göndermek: GraphQL subscription'ları ve WebSocket üzerinden STOMP
- Hepsini GraphQLTester ve bir STOMP istemcisiyle test etmek

Ön koşullar: Modül 06 (PostgreSQL), Modül 13 (Reactor Flux temelleri) · **Tahmini süre:** 5 saat · **Docker gerekir**

2. Kavramlar

2.1 Tek Sayfada GraphQL

REST'te her yanıtın şeklini **sunucu** belirler: GET /api/books/42 endpoint ne döndürüyorsa onu döndürür. GraphQL'de **tek bir endpoint** vardır (POST /graphql) ve **istemci**, tam olarak ihtiyaç duyduğu alanları adlandıran bir sorgu yazar:

```
{ book(isbn: "9780134685991") { title author { name } } }
```

```
{ "data": { "book": { "title": "Effective Java", "author": { "name": "Joshua Bloch" } } } }
```

İşlem	Amaç	Bu modüldeki taşıma
query	okuma	HTTP POST /graphql
mutation	değiştirme	HTTP POST /graphql
subscription	bir sonuç akışı	WebSocket /graphql

GraphQL, birçok farklı istemcinin (web, mobil, iş ortakları) aynı bağlantılı verinin farklı görünümüne ihtiyaç duyduğu durumlarda iyi bir seçimdir. Basit CRUD, dosya indirme veya HTTP önbellekleme için REST daha basit kalır.

2.2 Schema-First

Şema (src/main/resources/graphql/*.graphqls) sözleşmedir. Spring for GraphQL onu başlangıçta okur ve her alanı bir **data fetcher**'a bağlar. Anotasyonlu controller metotları data fetcher olur ve metot adı alan adıdır. Metodu olmayan bir alan, üst nesnenin aynı adlı özelliğinden okunur (Book.title() → title).

2.3 N+1 Problemi

Her birinin yazarıyla birlikte N kitaplık bir liste: Kitaplar için 1 sorgu, sonra yazarlar için N sorgu. Bir **DataLoader**, bir yürütme turunda istenen tüm anahtarları toplar ve onları birlikte yükler. @BatchMapping ile Spring for GraphQL DataLoader'ı sizin için kaydeder.

2.4 WebSocket ve STOMP

WebSocket açık kalan, çift yönlü bir bağlantıdır. Bu yüzden sunucu istediği an gönderebilir. WebSocket'in kendisi yalnızca frame taşır. **STOMP** üstüne küçük bir mesajlaşma protokolü ekler: SUBSCRIBE /topic/prices, SEND /app/..., MESSAGE. Spring'in **simple broker**'ı abonelikleri bellekte tutar ve her mesajı bir hedefin tüm abonelerine iletir.

	GraphQL subscription	STOMP
İstemcinin istediği	seçili alanlarıyla bir GraphQL sorgusu	bir hedef (/topic/prices)
Protokol	graphql-transport-ws	STOMP frame'leri
Uygun olduğu yer	zaten GraphQL konuşan istemciler	bildirimler, sohbet, dashboard'lar

3. Adım Adım Örnekler

Docker çalışırken uygulamayı başlatın. PostgreSQL kök compose.yaml'dan başlar ve Flyway dört kitap, üç yazar ve üç yorumla graphql şemasını oluşturur:

```
./mvnw -pl modules/17-graphql-websocket/lesson spring-boot:run
```

Tur çıktısı (kısaltılmış):

```
=== 3.2 A query: only the fields we ask for ===
  titles: [Designing Data-Intensive Applications, Effective Java, Java Puzzlers, Spring
↪   in Action]
=== 3.3 N+1: author via @BatchMapping, reviews via @SchemaMapping ===
  4 books → author queries: 1, review queries: 4
=== 3.4 A mutation, then one with a wrong number of stars ===
  pushed to the subscriber: Review[id=4, isbn=9780321336781, stars=4, text=Tricky!]
  added: {id=4, stars=4, text=Tricky!}
  error: [{message=Stars must be between 1 and 5, not 9, ...
↪   extensions={classification=BAD_REQUEST}}]
```

Ardından tarayıcı içi bir IDE için <http://localhost:8080/graphql>'i açın ve [requests.http](http://localhost:8080/graphql) içindeki istekleri çalıştırın.

3.1 Şema

```
type Query {
  books: [Book!]!
  book(isbn: ID!): Book
}

type Mutation {
  addReview(input: ReviewInput!): Review!
}

type Subscription {
  reviewAdded(isbn: ID!): Review!
}

type Book {
  isbn: ID!
  title: String!
  price: Float!
  author: Author!
  reviews: [Review!]!
}

type Author {
  id: ID!
  name: String!
}

type Review {
  id: ID!
  isbn: ID!
```

```

    stars: Int!
    text: String!
}

input ReviewInput {
    isbn: ID!
    stars: Int!
    text: String!
}

```

- ! null olamaz demektir. [Review!]! hiçbir zaman null olmayan ve null içermeyen bir listedir.
- book(isbn: ID!): Book'ta ! yoktur: Bilinmeyen bir ISBN hata değil, null döndürür.
- input ReviewInput argümanlar için bir tiptir. GraphQL'de çıktı tipleri (Review) ve girdi tipleri ayrılır.

Yapılandırma (application.yaml içinde spring: altında) GraphQL'i ve subscription'lar için WebSocket endpoint'ini etkinleştirir:

```

graphql:
  graphiql:
    enabled: true                # a browser IDE at /graphiql (development only)
  websocket:
    path: /graphql              # subscriptions over WebSocket, same path as HTTP

```

! Uyarı

GraphiQL'i yalnızca geliştirmede etkinleştirin. Production'da sorgu derinliğini ve karmaşıklığını sınırlamayı da düşünün: Bir istemci çok pahalı bir sorgu yazabilir.

3.2 Sorgular: @QueryMapping

```

@QueryMapping                                // Query.books
List<Book> books() {
    return catalog.findAllBooks();
}

@QueryMapping                                // Query.book(isbn: ID!) – null when
↪ unknown
@Nullable Book book(@Argument String isbn) {
    return catalog.findBook(isbn).orElse(null);
}

```

@QueryMapping, @SchemaMapping(typeName = "Query") için bir kısayoldur. @Argument aynı

adlı argümanı bağlar. Book record'unda isbn, title ve price vardır. Bu yüzden bu alanların metoda ihtiyacı yoktur.

HTTP üzerinden her istek, JSON gövdeli bir POST /graphql'dir. Bir alan başarısız olduğunda (örneğin bir resolver'daki doğrulama hatası) yanıt yine 200 OK'dir ve hatalar errors dizisindedir. GraphQLoverHttpTest, HttpGraphQLTester ile gerçek bir HTTP isteği gönderir.

3.3 İç İç Alanlar: @SchemaMapping ve @BatchMapping

Book'ta bir Author nesnesi yoktur, yalnızca authorId vardır. Book.author alanının bir data fetcher'a ihtiyacı vardır ve ders bunu verimli yoldan çözer:

```
/** Book.author for ALL books of the result in one call: no N+1. */
@BatchMapping
Map<Book, Author> author(List<Book> books) {
    Set<Long> ids = books.stream().map(Book::authorId).collect(Collectors.toSet());
    Map<Long, Author> authors = catalog.findAuthors(ids).stream()
        .collect(Collectors.toMap(Author::id, Function.identity()));
    return books.stream().collect(Collectors.toMap(Function.identity(), book ->
        ↪ authors.get(book.authorId())));
}
```

Metot sonucun **tüm** kitaplarını tek seferde alır ve kitap başına bir yazar döndürür. Repository tek bir IN sorgusuna ihtiyaç duyar:

```
/** One query for many authors: this is what @BatchMapping calls once per request. */
public List<Author> findAuthors(Collection<Long> ids) {
    authorQueries.incrementAndGet();
    return jdbc.sql("SELECT id, name FROM author WHERE id IN (:ids)")
        .param("ids", ids)
        .query(Author.class)
        .list();
}
```

Book.reviews farkın görünmesi için bilerek basit yolu kullanır:

```
/** Book.reviews, called once PER book: simple, but N+1 queries for a list of N books. */
@SchemaMapping
List<Review> reviews(Book book) {
    return catalog.findReviews(book.isbn());
}
```

BookQueryTest.batchMappingLoadsAllAuthorsInOneCallButReviewsNeedOneCallPerBook çağrılarını sayar: Bir yazar sorgusu, ama dört kitap için en az dört yorum sorgusu.

★ **İpucu**

@SchemaMapping tek bir nesnenin alanı için uygundur. Alan listelerde görüldüğü anda @BatchMapping kullanın.

3.4 Mutation'lar ve Hatalar

```
@MutationMapping
Review addReview(@Argument ReviewInput input) {
    return reviews.add(input);
}

@GraphQLExceptionHandler // like @ExceptionHandler, but produces
    ↳ a GraphQL error
GraphQLError invalidReview(InvalidReviewException e, DataFetchingEnvironment env) {
    return GraphQLErrorBuild-
        ↳ der.newError(env).errorType(ErrorType.BAD_REQUEST).message(e.getMessage()).build();
}

@GraphQLExceptionHandler
GraphQLError bookNotFound(BookNotFoundException e, DataFetchingEnvironment env) {
    return GraphQLErrorBuild-
        ↳ der.newError(env).errorType(ErrorType.NOT_FOUND).message(e.getMessage()).build();
}
```

- @Argument ReviewInput input, input nesnesini record'a bağlar.
- Servis normal Java exception'ları fırlatır:

```
public Review add(ReviewInput input) {
    if (input.stars() < 1 || input.stars() > 5) {
        throw new InvalidReviewException("Stars must be between 1 and 5, not " +
            ↳ input.stars());
    }
    if (catalog.findBook(input.isbn()).isEmpty()) {
        throw new BookNotFoundException(input.isbn());
    }
    Review review = catalog.insertReview(input);
    // two requests may add reviews at the same time: retry briefly instead of failing on
    ↳ a concurrent emit
    added.emitNext(review, Sinks.EmitFailureHandler.busyLooping(Duration.ofMillis(100)));
    return review;
}
```

- @GraphQLExceptionHandler, @ExceptionHandler'ın GraphQL karşılığıdır. Exception'ı yanıtın errors dizisinde, BAD_REQUEST veya NOT_FOUND gibi bir sınıflandırmayla bir

kayda çevirir. Handler'ı olmayan bir exception, genel bir mesajla `INTERNAL_ERROR` olur: Beklenmeyen hataların ayrıntıları istemciye verilmez.

Aynı handler'lar tüm controller'lara uygulansın diye bir `@ControllerAdvice` sınıfına konabilir.

3.5 Subscription'lar

Bir subscription metodu bir Flux döndürür. Her eleman istemci için bir sonuç olur:

```
@SubscriptionMapping
Flux<Review> reviewAdded(@Argument String isbn) {
    return reviews.reviewsAdded().filter(review -> review.isbn().equals(isbn));
}
```

Yorumlar, kodun içine gönderebildiği bir Flux olan bir Reactor **sink**'inden gelir:

```
// A hot stream: every subscriber gets the reviews added after it subscribed (no history,
↪ no buffer).
private final Sinks.Many<Review> added = Sinks.many().multicast().directBestEffort();
```

Bir abone yalnızca abone olduktan sonra eklenen yorumları alır. `ReviewSubscriptionTest` abone olur, iki kitap için yorum ekler ve yalnızca kendi kitabıninkileri alır.

i Not

Sink tek bir uygulama örneğinin belleğinde yaşar. Birden çok örnekte, A örneğine eklenen bir yorum B örneğinin abonelerine ulaşmaz. O zaman olayların bir broker'dan geçmesi gerekir (örneğin Kafka, modül 11 veya Redis pub/sub).

3.6 STOMP: Canlı Fiyat Bildirimleri

Yapılandırma WebSocket endpoint'ini ve broker'ı kaydeder:

```
@Configuration
@EnableWebSocketMessageBroker
class WebSocketConfiguration implements WebSocketMessageBrokerConfigurer {

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws"); // clients connect to
        ↪ ws://host:8080/ws
    }
}
```

```

@Override
public void configureMessageBroker(MessageBrokerRegistry broker) {
    broker.enableSimpleBroker("/topic");           // server → clients:
↪ /topic/...
    broker.setApplicationDestinationPrefixes("/app"); // clients →
↪ @MessageMapping methods: /app/...
}
}

```

Herhangi bir bean `SimpMessagingTemplate` ile bir hedefe gönderebilir. Burada normal bir REST endpoint'i bir fiyatı değiştirir ve tüm aboneleri bilgilendirir:

```

@PutMapping("/api/books/{isbn}/price")
@ResponseStatus(HttpStatus.NO_CONTENT)
void changePrice(@PathVariable String isbn, @Valid @RequestBody PriceChangeRequest
↪ request) {
    BigDecimal price = request.price();
    int updated = jdbc.sql("UPDATE book SET price = ? WHERE isbn = ?").params(price,
↪ isbn).update();
    if (updated == 0) {
        throw new ResponseStatusException(HttpStatus.NOT_FOUND, "No book with ISBN " +
↪ isbn); // → ProblemDetail
    }
    messaging.convertAndSend("/topic/prices", new PriceChanged(isbn, price)); // JSON
↪ to all subscribers
}

```

Tarayıcıda deneyin: `http://localhost:8080/prices.html`'i açın (@stomp/stompjs istemcili küçük bir sayfa), ardından `requests.http` içindeki fiyat değişikliğini çalıştırın. Yeni fiyat, sayfa yenilenmeden görünür.

`/app` öneki diğer yön içindir. Bir istemci `/app/...`'e bir mesaj gönderir ve bir `@Controller` içindeki `@MessageMapping` metodu onu işler. Ödevler yalnızca sunucu → istemci yönüne ihtiyaç duyar.

3.7 Test

- `@AutoConfigureGraphQLTester`, dokümanları HTTP olmadan doğrudan GraphQL motoruna karşı çalıştıran bir `GraphQLTester` verir. `path("books[*].title")` değerleri `Json-Path` ile seçer, `errors().expect(...)` hataları kontrol eder.
- `RANDOM_PORT` ile `@AutoConfigureHttpGraphQLTester` gerçek HTTP istekleri gönderir.
- `executeSubscription().toFlux(...)` `subscription`'ı bir `Flux` olarak döndürür. Onu `StepVerifier` ile kontrol edin.

- STOMP: `WebSocketStompClient` çalışan sunucuya bağlanır. SUBSCRIBE frame'i asenkronudur. Bu yüzden test, ilk mesaj gelene kadar eylemi `Awaitility` ile tekrarlar (`PriceNotificationTest`).

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Üst nesne başına çözülen bir alan, bir listenin her satırı için bir kez çalışır. 100 kitaplık bir liste 101 sorgu olmadan önce sorgu sayısını izleyin (buradaki testlerin yaptığı gibi).

- **Yapın:** Şemayı tabloların bir kopyası olarak değil, istemciler için tasarlayın. `authorId`: `ID!` yerine `author: Author!`.
- **Yapmayın:** İç exception mesajlarını istemcilere döndürmeyin. Bilinen exception'ları `@GraphQLExceptionHandler` ile eşleyin, geri kalanı `INTERNAL_ERROR` olsun.
- **Yapın:** "Bulunamadı" normal bir yanıt olduğunda alanı nullable yapın (`book(isbn): Book`).
- **Yapmayın:** Hatalar için HTTP durum kodlarına güvenmeyin. Alanlardaki hatalar 200 ve bir `errors` dizisiyle döner. Bu yüzden istemciler `errors`'ı her zaman kontrol etmelidir.
- **Yapın:** WebSocket endpoint'ini diğer her endpoint gibi güvenceye alın (modül 12) ve kimin hangi hedefe abone olabileceğini sınırlayın.
- **Yapmayın:** Simple broker'ın ve sink'in bellekte olduğunu unutmayın. Birden çok örnek için bir broker relay (RabbitMQ, ActiveMQ) veya bir mesajlaşma sistemi kullanın.

5. Özet

- Bir GraphQL şeması sözleşmedir. İstemci alanları seçer ve tek bir endpoint vardır: `/graphql`.
- `@QueryMapping`, `@MutationMapping` ve `@SubscriptionMapping` kök alanları gerçekleştirir. `@SchemaMapping` bir tipin alanını gerçekleştirir.
- `@BatchMapping` bir alanı tüm üst nesneler için tek çağrıda yükler ve N+1'i çözer.
- `@GraphQLExceptionHandler` exception'ları sınıflandırılmış GraphQL hatalarına çevirir.
- Subscription'lar bir Flux döndürür ve WebSocket üzerinden çalışır. STOMP bildirimler için hedefler ve bir broker ekler.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring for GraphQL Reference](#) · [Annotated Controllers](#) · [Testing](#)

- [Spring Boot — Spring for GraphQL](#)
- [Spring Framework — WebSockets and STOMP](#)
- [GraphQL — Learn](#)
- [STOMP 1.2 Specification](#)